

An Explainable Multi-Factor Pipeline for News Trust Assessment with Calibrated Scores and Honest Abstention

Ms. M. Kannukkiniyal

Guide, Assistant Professor

Department of Computer Science and Engineering

[kannukkiniyal.cse@kongu.edu]

Nandha Kumar S

23CSR140

Department of Computer Science and Engineering

[nandhakumars.23cse@kongu.edu]

Mugilan K S

23CSR138

Department of Computer Science and Engineering

[mugilanks.23cse@kongu.edu]

Nandhini A

23CSR141

Department of Computer Science and Engineering

[nandhinia.23cse@kongu.edu]

Course: 22CSP72 — Project Work II

GitHub Repository: [[github.com.Arumuganandhini](https://github.com/Arumuganandhini)]

Abstract— Most fake-news detectors just print “fake” or “real” from a black-box model, and a reader has no way to check that judgment or even understand what it is based on. This project builds a news trust assessment pipeline that breaks credibility down into six separate factors instead — how reliable the source is, whether the headline is clickbait, how biased or emotional the language is, whether the article names its sources, what persuasion tricks it uses, and whether other outlets are reporting the same facts. Each factor comes with its own evidence: the actual sentences from the article, which journalism checks it passed or failed, and links to other coverage of the same story. Two things make this different from a typical classifier project. One, the system abstains when it genuinely has no evidence — if a check cannot run, it does not just guess a middle score, it says so and shifts that weight onto the checks that did find something. Two, the final score is calibrated: a simple Platt-scaling step brought the expected calibration error down from 0.277 to 0.073 on the ISOT dataset, so a score of 70 actually means something close to 70% in practice. An earlier four-factor version of the pipeline reached 94.7% accuracy and 0.987 ROC-AUC on 300 ISOT articles with source leakage controlled for, compared to 82.0%/0.838 for a plain single-prompt baseline on the same model — this result is reported as historical since that model has since been withdrawn and two more factors were added afterward; re-running it on the current system is listed as pending work. The full system is deployed and works on live news, keeps working (in a reduced form) even if the AI provider goes down, and returns a complete report in about 11–19 seconds.

Index Terms— *explainable AI, misinformation detection, fact verification, model calibration, propaganda technique detection, graceful degradation, news trust assessment.*

I. INTRODUCTION

A. Background

There is simply too much news being published for any one person to fact-check by hand anymore. That is why automated help is appealing, and a lot of research already exists in this space. But almost all of it treats the problem as classification — feed in an article, get back a “fake” or “real” label.

B. Problem Statement

A plain label is not actually useful to a reader. If a system says an article is 87% likely to be fake, what is the reader supposed to do with that number? They cannot check it, cannot argue with it, cannot learn anything from it. It also hides which specific problem the article has — maybe the outlet has a bad track record, maybe the headline oversells the story, maybe it never names a source, maybe other outlets tell it differently. These need different reactions from the reader, and squashing them into one number throws that distinction away. On top of that, most of these models are overconfident: a model that says it is 90% sure is often only right 60% of the time, and almost none of the misinformation-detection papers we looked at even report a calibration number.

C. Motivation

So instead of trying to output a verdict, this project tries to produce a report: a score, the factors behind it, how much weight each one got, and the evidence for each one, quoting the article's own sentences back so a reader can actually verify what the system claims.

D. Research Gap

- Most systems give a label without any evidence a reader could actually check.
- Very few systems cross-check claims against other outlets, and the ones that do rarely mention how often they simply find nothing.
- Calibration is almost never reported, so a stated confidence score cannot really be trusted as a probability.
- Persuasion-technique detection exists as a research benchmark but nobody seems to have wired it into something a normal reader can use.
- No system we found says what it does when a component fails. In practice, the usual fallback — a neutral middle score — looks exactly the same to the reader as a check that actually ran and found nothing wrong.
- There isn't one deployed tool that combines source checking, headline analysis, language/bias analysis, transparency checks, persuasion detection, cross-source verification, and a calibrated score, all in one place.

E. Objectives

- Build a pipeline that turns a news article into a report with evidence attached, instead of a single label.
- Combine source, language, transparency, persuasion, and cross-source signals into one score that is still explainable.
- Make the system abstain honestly and redistribute weight when a check has nothing to go on, instead of faking a neutral score.

- Calibrate the final score so that a given confidence roughly matches how often it turns out to be right.
- Get the pipeline actually running on live news, with a fallback in case the AI provider is down.

F. Contributions

- A six-factor pipeline that works on real, live news articles and shows the evidence behind every factor.
- A cross-source verification step where the language model only extracts claims and judges stance on retrieved articles — it never answers from its own memory, and it abstains when there is nothing to check against.
- Honest abstention with weight redistribution, which we show later (Section VII) is not just a nice-to-have but actually fixes a real bug we found during development.
- A fallback mode using simple lexicon/pattern checks when the language model can't be reached, so the report never silently degrades without saying so.
- A model-provider gateway so the app is not stuck if one provider goes down or a model gets deprecated — both actually happened while building this.
- A leakage-controlled evaluation on ISOT with an ablation study and reported calibration numbers.

To be clear, this system doesn't claim to know objective truth. Cross-source verification just checks whether other outlets are reporting the same thing; if nobody else has covered a claim yet, it is marked unverified, not false.

II. RELATED WORK

A. Explainable Detection

DEFEND [1] builds explanations by looking at which user comments line up with which parts of an article, which works well but needs a comment section to already exist — not much use for an article someone just sent you a link to. ExFake [3] does something similar, pulling in social context and outside evidence, but again depends on signals that only show up after an article has already spread. Our factors only need the article text itself plus a live search of other outlets, so a report is available the moment someone submits a link.

B. Classification on ISOT

Ahmed et al. [2] put together the ISOT dataset along with n-gram baselines. A lot of papers get very high accuracy on ISOT, but that is partly because of a quirk in the dataset — the real articles all come from Reuters, so a model can basically just learn to recognize the outlet instead of learning anything about the content. We treat that as a bug in the benchmark rather than something to celebrate, and control for it directly (Section VI-B).

C. LLM-Based Verification

Wang et al. [4] use an LLM to weigh conflicting crowd reports about a claim, which needs a decent number of reports for each claim to work well. We use the model in a much narrower way — pull out the claims, then judge stance only against articles we actually retrieved — so the model is never the thing deciding what's true, retrieval is.

D. Evidence Retrieval

FEVER [5] set the standard for claim verification against retrieved evidence, but it uses Wikipedia as the corpus, which doesn't really work for breaking news that Wikipedia hasn't caught up on yet. We retrieve from live news coverage instead, which means we end up saying "unverified" a lot more often — we think that's an honest trade-off worth reporting rather than hiding.

E. Propaganda Technique Detection

Da San Martino et al. [7] defined the SemEval-2020 Task 11 taxonomy of propaganda/persuasion techniques, labelled at the span level across fourteen categories. As far as we can tell this has mostly stayed a research benchmark. We turned it into an actual reader-facing factor: it names the technique and quotes the exact sentence where it shows up.

F. Calibration

Guo et al. [6] showed that modern neural networks tend to be overconfident and tested a few fixes, including Platt scaling [8]. We apply that same idea to our trust score and actually report the before/after ECE numbers, which most misinformation papers in this space don't bother doing.

G. Where This Fits

Basically, most prior work is built around classification and, apart from [1] and [3], doesn't show its evidence. None of it says what happens when a component fails. That's the gap we're targeting — evidence, calibration, and a defined behaviour for partial failure.

III. PROBLEM FORMULATION AND RESEARCH GAP

This section lays out, more concretely, what prior work already does versus what we're adding on top of it.

A. What Prior Work Already Covers

- Explaining decisions using article + comment attention, but needing a comment stream to exist (dEFEND, ExFake).
- Weighing crowd reports to verify a claim, which needs many reports per claim to be reliable.
- Verifying claims against a static, curated corpus (FEVER uses Wikipedia).
- Labelling propaganda techniques at span level, but as a benchmark task, not inside a working reader-facing tool.
- Showing that neural classifiers are overconfident, and testing calibration fixes, but rarely applying this to misinformation scoring.

B. What We Add

- Six factors that all quote their own evidence, computed straight from the article plus live retrieval — no comment section needed.
- Cross-source verification against live news instead of a fixed corpus, with an explicit abstain state when nothing turns up.
- Honest abstention with weight redistribution across the remaining factors (Section IV-C).
- A deterministic fallback that still gives a usable result when the language model is unreachable.

- A provider-independent gateway that survives model deprecation and API congestion.
- A reported calibration number (ECE) for the combined score.

Two things worth clarifying so the claims here don't get overstated. First, when a factor abstains, that's not the system saying "this part is fine" — it just means it had no evidence either way. Second, the trust score is meant to help a reader decide how much scrutiny to apply, not a ruling on objective truth; cross-source verification only checks agreement between outlets, not ground truth.

IV. PROPOSED METHODOLOGY

A. System Overview

A user submits an article as a URL, or it comes in through a news feed. The article-reader module fetches the actual publisher page rather than relying on the news API's snippet, because that snippet is only around 200 characters — nowhere near enough for any factor that needs to reason about tone or sourcing. Before the fetched page is trusted, it has to pass a title-consistency check (Section V-A), since page scraping sometimes grabs the wrong content entirely. Once the text is ready, all six factors run at the same time and get combined by a weighted sum (Section IV-C) into the final report. Underneath everything sits a gateway to the language model (Section IV-D), and a cache keyed on the article URL so the same article isn't re-analysed every time someone opens it.

B. The Six Factors

1) Source reputation (weight 0.15): checks the outlet against a hand-curated list of 90 outlets with a factual-reporting rating, a lean, and an outlet type. Matching is done by tokenising names and domains and requiring a whole-word or prefix match — plain substring matching turned out to cause false matches (Section V-B).

2) Headline quality (0.10): looks for clickbait signals — curiosity-gap phrasing, over-the-top verbs, vague superlatives, listicle-style headlines — and reports exactly which signals it found, not just a number.

3) Language / bias (0.15): goes sentence by sentence looking for political bias, one-sided framing, emotionally loaded wording, or opinion dressed up as fact. Every flagged sentence gets quoted back exactly, along with what type of issue it is and why.

4) Source transparency (0.18): runs a five-point checklist — does it name sources, use attributed quotes, point to primary evidence, give concrete detail, and phrase claims as measured rather than absolute — scored out of ten, with a penalty for vague phrases like "experts say". This one doesn't depend on outside evidence at all, so it's still useful for a story that literally nobody else has covered yet, which is exactly when cross-source verification has nothing to say.

5) Persuasion technique (0.15): checks for techniques from the SemEval-2020 Task 11 list [7], trimmed down to fourteen categories that a non-expert reader can actually recognise. Every detection quotes the exact span it applies to, and we cap it at three per article so the report doesn't get cluttered.

6) Cross-source verification (0.27): pulls out the claims in the article, then searches for coverage of each one from other outlets and checks whether they agree, disagree, or say nothing. Each claim

ends up marked supported, contradicted, or unverified, with links to whichever articles were used.

C. Combining Scores, and What Happens When a Factor Has Nothing

Say F is the set of all six factors, w_e is factor f 's normal weight (they all add up to 1), and s_e is its score out of 10. Let I be the subset of factors that actually found something to work with — a factor counts as “uninformative” if it had no evidence at all (for cross-source verification, that means no other outlet had covered any of the claims). Whenever a factor is uninformative, it drops out and its weight gets shared proportionally among the ones that did produce a result:

$$w_e' = w_e / (\sum_{\{g \in I\}} w_g) \text{ for } f \in I, \quad w_e' = 0 \text{ otherwise}$$

$$S = \sum_{\{f \in I\}} w_e' \cdot s_e$$

On the report page, a factor that abstains just shows a hatched bar with no number and a short written reason, e.g. “Not counted: no other outlet has covered this story yet.” Since the weight percentages are rounded for display, we use the largest-remainder method so they always add up to exactly 100 on screen. The final score maps to a verdict: 7.5+ is “Looks trustworthy”, 5.5+ is “Mostly fine”, 4.0+ is “Read carefully”, and anything below that is “Be skeptical”.

D. Not Locking Ourselves Into One Model Provider

Every factor talks to the language model through one shared gateway that handles concurrency limits, retries, and parsing the structured output. The provider and the exact model are just config values, and the gateway reacts differently depending on what actually failed: if a model comes back 404/410 (i.e. it's been pulled), the gateway switches permanently to whatever's next in the config; if it's a 429/503 (rate-limited or overloaded), it retries the same model with backoff and respects Retry-After instead of panicking and demoting a model that's otherwise fine; and if a whole provider is exhausted, it falls through to the next one. Both of these actually happened during development, which is the whole reason this exists.

E. Degrading Gracefully Instead of Breaking

If nothing answers at all, each content factor switches to a simple lexicon/pattern-based version instead of just failing. The report clearly marks itself as degraded, both at the top and next to each affected factor, so nobody mistakes a weaker fallback analysis for the full thing (results in Section VII-D).

V. SYSTEM IMPLEMENTATION

Backend is Node.js/Express with MongoDB, frontend is React, news comes from NewsAPI, and the language model is either NVIDIA NIM or Google Gemini depending on config.

A. Getting the Actual Article Text

Fetching the publisher's own page instead of using NewsAPI's ~200-character snippet gets us 2,000–6,000 characters, and only costs 1–3 seconds with no model call involved. But scraping isn't always reliable — we had a case where an AP article's page returned nav-bar and newsletter-signup text instead of the actual story, and the resulting summary confidently talked about a completely unrelated trade dispute. So now we only accept the scraped text if at least half of the headline's distinctive words show up in it; otherwise

we just fall back to the feed snippet. Out of ten test articles, seven passed this check.

B. Matching Outlets Correctly

Our first version of outlet matching used plain substring search, which backfired — it matched “Yahoo Entertainment” against a database entry for “RT” because “rt” shows up inside the string. We fixed this by tokenising both sides and requiring an exact or whole-word prefix match, done in two passes.

C. Dealing With Broken JSON From the Model

We ran into three different ways the model's JSON output could break: wrapping it in markdown fences or extra prose, small syntax errors like unquoted strings or trailing commas, and outright truncation. For truncation specifically, the parser tries to salvage whatever elements did complete instead of throwing the whole response away. There was also a weird bug where the NewsAPI's “[+8026 chars]” truncation marker would get echoed back by the model and picked up as the start of a JSON array — we just strip that marker out before building the prompt now.

D. Caching So We're Not Re-Analysing the Same Article

Reports get cached by article URL for 14 days. Each cached report also carries a schema version, so if we change the pipeline, old cached reports get re-analysed instead of silently being served with a mismatched structure. There's also a pre-warm script that analyses a whole news category ahead of time.

VI. EXPERIMENTAL METHODOLOGY

A. Dataset

We used ISOT [2]: after filtering out articles that were too short, we had 43,294 usable articles (21,915 fake, 21,379 real). Sampling is stratified and seeded, and the exact same sample is reused across every configuration we tested.

B. Fixing the Reuters Leakage Problem

Since all of ISOT's real articles come from Reuters, a model can cheat by just recognising the outlet instead of reading the content. We handle this two ways: hiding the source identity from the pipeline during evaluation, and stripping the “CITY (Reuters) —” prefix that shows up at the start of Reuters articles. So the numbers we report are based on content only. Cross-source verification is turned off for these dataset runs, since old articles obviously have no live coverage to check against — in production this would just show up as an abstained factor anyway, so nothing changes behaviourally.

C. Protocol and Metrics

We fit the threshold and the Platt-scaling parameters on one half of the data (validation) and report every metric on the other, unseen half (test). Metrics: accuracy, macro-F1, ROC-AUC, and 10-bin expected calibration error. The baseline is a single big prompt asking the same underlying model to rate credibility directly — basically the “just ask the LLM” approach. If an API call fails during evaluation, we retry that one article instead of recording a fallback score, so network flakiness can't sneak into the results.

VII. RESULTS AND DISCUSSION

A. Classification and Ablation (Older Configuration)

This subsection is a bit different — it’s a measurement we took earlier on a model that the provider has since pulled, and before we’d even built the transparency and persuasion factors. We’re keeping it because it’s the result that convinced us the multi-factor approach was worth building, but it should not be read as a claim about the current six-factor system. Re-running this on the current pipeline is listed as immediate future work (Section IX-A). Setup: $n = 300$, stratified 150 fake / 150 real, seed 42, model meta/llama-3.1-8b-instruct, four-factor pipeline at the time:

Configuration	Acc.	Macro-F1	AUC	ECE raw	ECE cal.
Single-prompt baseline	82.0%	81.9%	0.838	0.215	0.151
Pipeline (clickbait+bias)	94.7%	94.7%	0.987	0.277	0.073
Ablation: clickbait only	93.3%	93.3%	0.975	0.339	0.118
Ablation: bias only	92.7%	92.7%	0.934	0.359	0.204

TABLE I. Historical classification and ablation results on ISOT.

Splitting the judgment into separate factors beat a single big prompt on the same model by 12.7 accuracy points and 0.149 AUC. The ablation results also line up sensibly — bias-only did worst, clickbait-only did better, and combining both did best of all — so both factors are actually pulling their weight.

B. Calibration

The raw scores rank articles correctly for the most part, but they bunch up in the middle of the range, which gives an ECE of 0.277. After fitting Platt scaling [8] on the validation half, that drops to 0.073, which is well within the range people usually call “well calibrated.” In practice this means a 70% trust score is actually close to being right 70% of the time.

C. How Much Article Text Is Actually Useful

Turns out feeding the model more article text doesn’t automatically help, because the reply budget has to cover both the reasoning and the required JSON output. We measured this on a reasoning model with a fixed token budget:

Input chars	Short summary	Detailed summary
800	6.8 s, 361 chars	3.6 s, 504 chars
1,500	9.6 s, 666 chars	32.7 s, empty
3,000	32.8 s, no JSON	23.9 s, empty

TABLE II. Effect of input length on structured-output reliability.

Past around 800 characters, the model was running out of budget before it could actually finish the JSON. So we cap the text fed into the factors and summarisers at 900 characters, which is still four to five times bigger than what NewsAPI’s snippet gives us.

D. Fallback Mode Without the Language Model

Article	Score (0 model calls)	Verdict
Fabricated, loaded language	2.3 / 10	Be skeptical
Ordinary attributed reporting	9.8 / 10	Looks trustworthy

TABLE III. Deterministic fallback with the language model unreachable.

Even with zero model calls, the fallback still tells the two articles apart clearly, catching loaded language, name-calling, bandwagon appeals, and appeal-to-fear, plus four unattributable phrases. This section is also where we log a bug we found and fixed: four of the content factors used to respond to a failed model call with a neutral

5/10 and a “failed” flag that the aggregator never actually checked. So a full outage would silently produce a confident-looking “Mostly fine, ~5.5/10” report made entirely of placeholder scores — which is basically research gap 5 from Section I-D showing up in our own code, and it’s the whole reason we built honest abstention in the first place.

E. Checking That the Evidence Is Actually Real

Since every factor quotes the article directly, we could just check whether those quotes are real. We built a test article on purpose with both legit facts and obviously loaded language in it, and checked all nine quoted spans — four flagged sentences, three persuasion techniques with their quotes, one vague-attribution phrase, and the named sources — against the source text. All nine matched word for word. The transparency checklist passed 4 out of 5 checks and correctly failed the “claims are measured” check, pointing to the sentence “Everyone knows parking is already impossible downtown.” Cross-source verification abstained here, which makes sense since the test article doesn’t exist anywhere else. A proper live-news study with manual annotation is still pending (Section IX-A).

F. How Fast the Deployed System Actually Is

Property	Phase I	Current
Cold analysis, one article	~60 s (to 116 s)	11–19 s
Cached analysis	full recomputation	~5 ms
First factor visible	on completion	< 1 s
Backend tests passing	—	42 / 42

TABLE IV. System latency, Phase I vs. current deployment.

We learned the hard way that different providers handle concurrency very differently, so the gateway now sets its concurrency limit based on whichever provider is currently active instead of assuming they all behave the same. The overall speed-up above comes from four separate changes: picking a faster provider, making concurrency provider-aware, running per-claim verification in parallel, and overlapping claim extraction with the other content factors instead of doing everything one after another.

G. Discussion

The bug in Section VII-D is honestly the best argument for this whole design. A neutral fallback score isn’t just imprecise, it actively misleads people in one direction, because a middling number on a 0–10 scale reads as “eh, probably fine.” We think any system that stitches together several components that can each fail on their own needs to define what happens on failure as part of the spec, not leave it as an implementation detail nobody thought about.

We also tried to be deliberate about what the model is and isn’t allowed to decide. It only ever judges things that are linguistic — is this sentence loaded, does this article name a source — and never judges whether a claim is actually true; that weight sits entirely on retrieval. That’s why cross-source verification abstains so often, and we’d rather report that honestly than quietly engineer the abstention rate down to look better.

One thing we actually built and then removed: a seventh factor that checked claims against professional fact-checker sites like Snopes. It worked — given a claim Snopes had already rated, it would pull the page, read the “False” rating out of the ClaimReview markup, and cite the URL, and it correctly refused to match unrelated claims on the same topic. It’s not in the live system right

now because Google's Fact Check Tools API needs billing enabled, the keyless workaround eats into a request budget that our article feed also depends on, and scraping fact-checker sites directly doesn't work since their pages render with JavaScript. The code is still there, just switched off, in case we revisit it.

VIII. LIMITATIONS

- ISOT is 2016–2017 US political news, so we don't actually know how well this generalises to current news.
- Stance judgment currently works off headlines and descriptions rather than full article bodies; swapping in a proper NLI model would be a fairly easy upgrade.
- Any bias baked into the underlying language model carries over into our language/bias factor — low temperature and structured output cut down on randomness, not on bias.
- The outlet database only covers 90 sources, so smaller or regional publishers often get no reputation score at all, which is the most common gap we've seen in real use.
- The Section VII-A numbers are from a single dataset and a single (now-retired) model, and will be replaced once the current-system re-run (Section IX-A) is done.
- The live-news verification study (Section VII-E) hasn't been completed yet.

IX. FUTURE WORK

A. *Right Now*

Re-run the full ISOT benchmark on the current six-factor pipeline with the current model, including ablations for the transparency and persuasion factors, to replace Section VII-A. Also finish the live-news verification study from Section VII-E.

B. *Later*

A reading-trust history view for users — since per-article scores are already stored, this is mostly just a visualisation problem, not a new pipeline. Also, letting the source-reputation database grow itself: because every analysis already produces a per-outlet score, we could accumulate ratings for outlets that aren't in the curated list yet, and check those against the hand-curated ones to make sure they're sane.

X. CONCLUSION

We built a six-factor pipeline for assessing news trustworthiness that shows its evidence instead of just handing out a label, reports a calibrated confidence, and — the part we care about most — abstains honestly when a check has nothing to go on, instead of quietly faking a neutral score. An earlier four-factor version beat a single-prompt baseline on the same model by 12.7 accuracy points, and Platt scaling brought the calibration error down from 0.277 to 0.073. The deployed system keeps working through provider outages and model deprecations, falls back to a simpler analysis without pretending it's the full one, and returns a complete report in 11–19 seconds.

REFERENCES

- [1] K. Shu, L. Cui, S. Wang, D. Lee, and H. Liu, “dEFEND: Explainable Fake News Detection,” in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2019, pp. 395–405.
- [2] H. Ahmed, I. Traore, and S. Saad, “Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques,” in Proc. Int. Conf. Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments (ISDDC), 2017, pp. 127–138.
- [3] S. Amri, D. Sallami, and E. Aïmeur, “ExFake: Towards an Explainable Fake News Detection Based on Content and Social Context Information,” 2023.
- [4] B. Wang, J. Ma, H. Lin, Z. Yang, R. Yang, Y. Tian, and Y. Chang, “Explainable Fake News Detection with Large Language Model via Defense Among Competing Wisdom,” in Proc. ACM Web Conference (WWW), 2024.
- [5] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a Large-scale Dataset for Fact Extraction and VERification,” in Proc. NAACL-HLT, 2018, pp. 809–819.
- [6] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 1321–1330.
- [7] G. Da San Martino, A. Barrón-Cedeño, H. Wachsmuth, R. Petrov, and P. Nakov, “SemEval-2020 Task 11: Detection of Propaganda Techniques in News Articles,” in Proc. 14th Workshop on Semantic Evaluation (SemEval), 2020, pp. 1377–1414.
- [8] J. Platt, “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods,” in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.